

M1 Mathématiques – Informatique pour Mathématiciens

Séance 3 : POO Avancée
Structures Algébriques – Polynômes et Matrices

M1 MAFI – Université d’Antananarivo

Année 2026–2027

Contents

1 Préambule

Ressources recommandées

- Cormen, Leiserson, Rivest, Stein – *Introduction to Algorithms*, 4e éd., MIT Press, 2022 (chapitres 2, 4, 33 pour la complexité et les structures de données).
- Exo7 – *Algorithmes et mathématiques* – <http://exo7.emath.fr> (algorithmique sur les polynômes et les matrices).
- Avigad, Moura, Kong, Ullrich – *Theorem Proving in Lean 4*, https://leanprover.github.io/theorem_proving_in_lean4/ (formalisation).
- Avigad et al. – *Mathematics in Lean*, https://leanprover-community.github.io/mathematics_in_lean/ (polynômes en Lean : `Mathlib.RingTheory.Polynomial`).
- Strang, G. – *Computational Science and Engineering*, Wellesley–Cambridge Press, 2007 (matrices, élimination gaussienne, complexité numérique).
- Trefethen, L. N. & Bau, D. – *Numerical Linear Algebra*, SIAM, 1997 (décomposition LU, stabilité numérique).

L’IA, partenaire et non substitut

Lors des séances précédentes, vous avez pris en main Python et les paradigmes de base de la programmation orientée objet. Cette séance franchit un cap : modéliser des *structures algébriques* (anneaux, espaces vectoriels) par des classes Python, puis en formaliser les propriétés en Lean 4.

Un assistant IA peut vous aider à écrire du code, mais il ne peut ni garantir que votre classe `Polynome` respecte l’axiomatique d’un anneau, ni prouver formellement que l’algorithme d’Euclide pour polynômes termine en $\mathcal{O}(n^2)$ opérations. Ces responsabilités vous appartiennent.

2 Objectifs de la séance

Objectifs de la séance

1. Maîtriser la **POO avancée** en Python : héritage, classes abstraites (ABC), surcharge complète des opérateurs (*dunder methods*).
2. Implémenter la classe `Polynome` reflétant la structure d’anneau $\mathbb{K}[X]$: addition, multiplication, évaluation par la méthode de Horner, dérivée formelle, et PGCD par l’algorithme d’Euclide.
3. Implémenter la classe `Matrice` : opérations arithmétiques, élimination de Gauss–Jordan pour la résolution de $Ax = b$, calcul du déterminant.
4. Effectuer une **introduction légère à Lean 4** : formaliser une propriété élémentaire des polynômes (degré de la somme).
5. Analyser et mesurer empiriquement les complexités de Horner ($\mathcal{O}(n)$), de la multiplication naïve de polynômes ($\mathcal{O}(n^2)$) et de Gauss ($\mathcal{O}(n^3)$).

3 POO Avancée : Héritage, ABC et Opérateurs

3.1 Introduction

La Programmation Orientée Objet (POO) s'avère particulièrement naturelle pour les mathématiques : une classe Python peut modéliser un objet algébrique, ses méthodes représentent les opérations, et les propriétés axiomatiques (commutativité, distributivité...) se vérifient par des tests automatisés ou une preuve formelle.

Dans les séances précédentes, vous avez créé des classes simples (`Rationnel`, `GrandEntier`). Nous allons maintenant exploiter des mécanismes plus puissants.

3.2 Classes abstraites (ABC)

Définition 3.1 (Classe abstraite). Une **classe abstraite** est une classe qui ne peut pas être instanciée directement. Elle définit une *interface* (ensemble de méthodes obligatoires) que les sous-classes doivent implémenter. En Python, on utilise le module `abc`.

```
1 from abc import ABC, abstractmethod
2
3 class StructureAlgebrique(ABC):
4     """
5     Classe abstraite definissant l'interface d'un anneau.
6     Toute sous-classe doit implementer __add__, __mul__ et zero
7     """
8
9     @abstractmethod
10    def __add__(self, other):
11        """Addition dans la structure."""
12        pass
13
14    @abstractmethod
15    def __mul__(self, other):
16        """Multiplication dans la structure."""
17        pass
18
19    @property
20    @abstractmethod
21    def zero(self):
22        """Element neutre pour l'addition."""
23        pass
24
25    @abstractmethod
26    def __eq__(self, other):
27        """Egalite de deux elements."""
28        pass
29
30    def __ne__(self, other):
31        return not self.__eq__(other)
```

Listing 1: Interface d'une structure algébrique

Exemple 3.1 (Pourquoi une interface ?). En forçant toute sous-classe à implémenter `__add__` et `zero`, on garantit que les algorithmes génériques (Euclide, évaluation) s'appliquent indistinctement aux polynômes, aux matrices ou aux entiers, simplement en passant le bon type en argument.

3.3 Héritage et polymorphisme

```

1 class Animal:
2     def __init__(self, nom):
3         self.nom = nom
4
5     def parler(self):
6         raise NotImplementedError
7
8 class Chien(Animal):
9     def parler(self):
10        return "Ouaf!"
11
12 class Chat(Animal):
13     def parler(self):
14        return "Miaou!"
15
16 # Polymorphisme : meme appel, comportement different
17 for animal in [Chien("Rex"), Chat("Minou")]:
18     print(f"{animal.nom} dit : {animal.parler()}")
19 # Rex dit : Ouaf!
20 # Minou dit : Miaou!

```

Listing 2: Exemple d'héritage simple

Remarque 3.1. En mathématiques, c'est l'équivalent du **polymorphisme** : une même opération $+$ s'applique aux entiers, aux rationnels, aux polynômes, etc.

3.4 Surcharge complète des opérateurs

Python permet de surcharger les opérateurs arithmétiques via les *méthodes spéciales* (*dunder methods*), résumées dans le tableau suivant :

Opérateur	Méthode	Opérateur	Méthode
$a + b$	<code>__add__</code>	$a == b$	<code>__eq__</code>
$a - b$	<code>__sub__</code>	$a < b$	<code>__lt__</code>
$a * b$	<code>__mul__</code>	<code>len(a)</code>	<code>__len__</code>
$-a$	<code>__neg__</code>	<code>str(a)</code>	<code>__str__</code>
$a ** n$	<code>__pow__</code>	<code>repr(a)</code>	<code>__repr__</code>

4 Polynômes : Classe et Algorithmes

4.1 Introduction

L'anneau des polynômes $\mathbb{K}[X]$ (avec $\mathbb{K} = \mathbb{R}$ ou $\mathbb{Q}$) est omniprésent en mathématiques : interpolation, cryptographie, théorie des codes, analyse numérique. Implémenter une classe `Polynome` rigoureuse est une excellente mise en pratique de la POO et de l'arithmétique algorithmique.

4.2 Définition et représentation

Définition 4.1 (Polynôme). Un polynôme à coefficients dans $\mathbb{K}$ est une expression formelle

$$P = a_0 + a_1X + a_2X^2 + \cdots + a_nX^n, \quad a_i \in \mathbb{K}.$$

Son **degré** est $\deg(P) = \max\{i : a_i \neq 0\}$ (par convention $\deg(0) = -\infty$).

Nous représentons P par la liste de ses coefficients ordonnée par degré croissant : $[a_0, a_1, \dots, a_n]$ avec $a_n \neq 0$.

```
1 from abc import ABC, abstractmethod
2 import math
3
4 class Polynome(StructureAlgebrique):
5     """
6     Représentation d'un polynome par ses coefficients.
7     coeffs[i] est le coefficient de X^i.
8     """
9
10    def __init__(self, coeffs):
11        """
12        coeffs : liste ou tuple de coefficients (degré
13        croissant).
14        Exemple : Polynome([1, 2, 3]) représente 1 + 2X + 3X^2.
15        """
16        self.coeffs = list(coeffs)
17        self._normaliser() # supprime les zeros de tete
18
19    def _normaliser(self):
20        """Supprime les coefficients dominants nuls."""
21        while len(self.coeffs) > 1 and self.coeffs[-1] == 0:
22            self.coeffs.pop()
23
24    @property
25    def degree(self):
26        """Retourne le degré du polynome (-1 pour le polynome
27        nul)."""
28        if self.coeffs == [0]:
29            return -1
30        return len(self.coeffs) - 1
31
32    @property
```

```

31     def zero(self):
32         return Polynome([0])
33
34     def __len__(self):
35         return self.degre + 1
36
37     def __getitem__(self, i):
38         """Acces au coefficient de X^i."""
39         if 0 <= i < len(self.coeffs):
40             return self.coeffs[i]
41         return 0
42
43     def __eq__(self, other):
44         if isinstance(other, (int, float)):
45             return self.coeffs == [other]
46         return self.coeffs == other.coeffs
47
48     def __repr__(self):
49         return f"Polynome({self.coeffs})"
50
51     def __str__(self):
52         if self.degre == -1:
53             return "0"
54         termes = []
55         for i, c in enumerate(self.coeffs):
56             if c == 0:
57                 continue
58             if i == 0:
59                 termes.append(str(c))
60             elif i == 1:
61                 termes.append(f"({c})*X" if c != 1 else "X")
62             else:
63                 termes.append(f"({c})*X^{i}" if c != 1 else f"X
64 ^{i}")
65         return " + ".join(reversed(termes)) if termes else "0"

```

Listing 3: Classe Polynome : initialisation et affichage

4.3 Opérations arithmétiques

Proposition 4.1 (Addition et multiplication dans $\mathbb{K}[X]$).]Soient $P = \sum_{i=0}^m a_i X^i$ et $Q = \sum_{j=0}^n b_j X^j$.

$$(P + Q) = \sum_{k=0}^{\max(m,n)} (a_k + b_k) X^k, \quad \text{avec } a_k = 0 \text{ si } k > m.$$

$$(P \cdot Q) = \sum_{k=0}^{m+n} \left(\sum_{i+j=k} a_i b_j \right) X^k.$$

Proof. Résulte directement de la distributivité et de $X^i \cdot X^j = X^{i+j}$. □

```

1  def __add__(self, other):
2      """Addition de deux polynomes. Complexite : O(max(deg P
   , deg Q))."""
3      n = max(len(self.coeffs), len(other.coeffs))
4      resultat = [self[i] + other[i] for i in range(n)]
5      return Polynome(resultat)
6
7  def __neg__(self):
8      """Oppose d'un polynome."""
9      return Polynome([-c for c in self.coeffs])
10
11 def __sub__(self, other):
12     """Soustraction de deux polynomes."""
13     return self + (-other)
14
15 def __mul__(self, other):
16     """
17     Multiplication de deux polynomes (algorithme naif).
18     Complexite : O(deg(P) * deg(Q)) = O(n^2).
19     """
20     if isinstance(other, (int, float)):
21         return Polynome([c * other for c in self.coeffs])
22     m, n = self.degree, other.degree
23     if m == -1 or n == -1:
24         return Polynome([0])
25     resultat = [0] * (m + n + 1)
26     for i, ai in enumerate(self.coeffs):
27         for j, bj in enumerate(other.coeffs):
28             resultat[i + j] += ai * bj
29     return Polynome(resultat)
30
31 def __rmul__(self, scalaire):
32     return self.__mul__(scalaire)
33
34 def __pow__(self, n):
35     """Exponentiation rapide : O(deg(P) * n * log n)."""
36     if n == 0:
37         return Polynome([1])
38     if n % 2 == 0:
39         demi = self ** (n // 2)
40         return demi * demi
41     return self * (self ** (n - 1))

```

Listing 4: Addition, soustraction, négation, multiplication

4.4 Évaluation : méthode de Horner

Définition 4.2 (Méthode de Horner, 1819). Pour un polynôme $P = a_0 + a_1X + \dots + a_nX^n$, on réécrit :

$$P(x) = a_0 + x(a_1 + x(a_2 + \dots + x(a_{n-1} + x \cdot a_n) \dots)).$$

Ceci permet d'évaluer $P(x)$ en exactement n additions et n multiplications, sans aucun calcul de puissance.

Proposition 4.2 (Complexité de Horner). *La méthode de Horner évalue P en x en $\mathcal{O}(n)$ opérations arithmétiques, contre $\mathcal{O}(n^2)$ pour l'évaluation terme à terme naïve.*

Proof. L'algorithme effectue exactement n additions et n multiplications : une de chaque par coefficient, traitée de a_n vers a_0 . □ □

```

1  def evaluer(self, x):
2      """
3      Evaluation de P(x) par la methode de Horner.
4      Complexite : O(n) ou n = deg(P).
5
6      Reference : Knuth, TAOCP vol. 2, section 4.6.4.
7      """
8      resultat = 0
9      for coeff in reversed(self.coeffs):
10         resultat = resultat * x + coeff
11     return resultat
12
13     def derivee(self):
14         """
15         Derivee formelle de P.
16         Si P = sum a_i X^i alors P' = sum i*a_i X^{i-1}.
17         Complexite : O(n).
18         """
19         if self.degre <= 0:
20             return Polynome([0])
21         return Polynome([i * self.coeffs[i]
22                          for i in range(1, len(self.coeffs))])

```

Listing 5: Évaluation et dérivée formelle

Exemple 4.1 (Application de Horner). Soit $P = 3X^3 - 2X^2 + X - 5$. Pour $x = 2$:

$$\begin{aligned}
 P(2) &= -5 + 2(1 + 2(-2 + 2 \cdot 3)) \\
 &= -5 + 2(1 + 2 \cdot 4) = -5 + 2 \cdot 9 = 13.
 \end{aligned}$$

```

1  P = Polynome([-5, 1, -2, 3])    # -5 + X - 2X^2 + 3X^3
2  print(P.evaluer(2))             # 13
3  print(P.derivee())              # 1 + (-4)*X + (9)*X^2

```

Listing 6: Test de la méthode de Horner

4.5 PGCD de polynômes : algorithme d'Euclide

Théorème 4.1 (Division euclidienne dans $\mathbb{K}[X]$). *Pour tout $A, B \in \mathbb{K}[X]$ avec $B \neq 0$, il existe un unique couple (Q, R) tel que $A = BQ + R$ et $\deg(R) < \deg(B)$.*

Proposition 4.3 (Algorithme d'Euclide pour $\mathbb{K}[X]$). *L'algorithme suivant calcule $\text{pgcd}(A, B)$ en au plus de $\deg(A) + \deg(B)$ étapes, chaque étape réduisant strictement le degré du reste.*

```

1  def division_euclidienne(self, diviseur):
2      """
3      Retourne (Q, R) tels que self = diviseur * Q + R, deg(R)
4      ) < deg(diviseur).
5      Algorithme : division longue.
6      Complexite : O(deg(self) * deg(diviseur)).
7      """
8      if diviseur == Polynome([0]):
9          raise ZeroDivisionError("Division par le polynome
10 nul")
11     dividende = Polynome(self.coeffs[:])
12     quotient = Polynome([0])
13     while dividende.degre >= diviseur.degre:
14         # terme dominant du quotient
15         deg_diff = dividende.degre - diviseur.degre
16         coeff_div = dividende.coeffs[-1] / diviseur.coeffs
17         [-1]
18         # monome : coeff_div * X^deg_diff
19         monome_coeffs = [0] * deg_diff + [coeff_div]
20         monome = Polynome(monome_coeffs)
21         quotient = quotient + monome
22         dividende = dividende - (monome * diviseur)
23     return quotient, dividende
24
25 @staticmethod
26 def pgcd(A, B):
27     """
28     PGCD de A et B dans K[X] par l'algorithme d'Euclide.
29     Retourne un representant normalise (coefficient
30 dominant = 1).
31     Complexite : O(n^2) ou n = max(deg A, deg B).
32     """
33     while B != Polynome([0]):
34         _, R = A.division_euclidienne(B)
35         A, B = B, R
36     # Normalisation (polynome unitaire)
37     lc = A.coeffs[-1]
38     return Polynome([c / lc for c in A.coeffs])

```

Listing 7: Division euclidienne et PGCD de polynômes

Exemple 4.2 (PGCD de polynômes). Soient $A = X^3 - 1$ et $B = X^2 - 1$. On a $\text{pgcd}(A, B) = X - 1$.

```

1  A = Polynome([-1, 0, 0, 1])    # X^3 - 1
2  B = Polynome([-1, 0, 1])       # X^2 - 1
3  print(Polynome.pgcd(A, B))     # X - 1 (attendu)

```

5 Matrices : Classe et Élimination de Gauss

5.1 Introduction

Les matrices sont l'objet central de l'algèbre linéaire numérique. Nous implémentons une classe `Matrice` avec les opérations de base, puis l'élimination de Gauss–Jordan pour la résolution de systèmes $Ax = b$ et le calcul du déterminant.

5.2 Classe Matrice

```
1 class Matrice:
2     """
3     Matrice m x n de nombres reels.
4     Stockage : liste de listes (lignes).
5     """
6
7     def __init__(self, lignes):
8         """
9         lignes : liste de listes de nombres.
10        Exemple : Matrice([[1, 2], [3, 4]]) cree une matrice 2
11        x2.
12        """
13        self.data = [list(ligne) for ligne in lignes]
14        self.m = len(self.data)          # nombre de lignes
15        self.n = len(self.data[0]) if self.data else 0 #
16        nombre de colonnes
17
18    @classmethod
19    def nulle(cls, m, n):
20        """Matrice nulle m x n."""
21        return cls([[0] * n for _ in range(m)])
22
23    @classmethod
24    def identite(cls, n):
25        """Matrice identite n x n."""
26        M = cls.nulle(n, n)
27        for i in range(n):
28            M.data[i][i] = 1
29        return M
30
31    def __getitem__(self, indices):
32        i, j = indices
33        return self.data[i][j]
34
35    def __setitem__(self, indices, valeur):
36        i, j = indices
37        self.data[i][j] = valeur
38
39    def __repr__(self):
40        return f"Matrice({self.data})"
```

```

40     def __str__(self):
41         largeur = max(len(f"{self.data[i][j]:.3g}")
42                        for i in range(self.m)
43                        for j in range(self.n))
44         lignes = []
45         for ligne in self.data:
46             row = " ".join(f"{x:{largeur}.3g}" for x in ligne)
47             lignes.append(f"| {row} |")
48         return "\n".join(lignes)

```

Listing 8: Classe Matrice : initialisation et affichage

```

1     def __add__(self, other):
2         """Addition. Complexite : O(mn)."""
3         assert self.m == other.m and self.n == other.n, "
Dimensions incompatibles"
4         return Matrice([[self.data[i][j] + other.data[i][j]
5                          for j in range(self.n)]
6                          for i in range(self.m)])
7
8     def __sub__(self, other):
9         return self + (-other)
10
11    def __neg__(self):
12        return Matrice([[ -x for x in ligne] for ligne in self.
data])
13
14    def __mul__(self, other):
15        """
16        Multiplication matricielle (algorithme naif).
17        Complexite : O(m * n * p) pour une matrice m x n fois n
x p.
18        """
19        if isinstance(other, (int, float)):
20            return Matrice([[other * x for x in ligne] for
ligne in self.data])
21        assert self.n == other.m, "Dimensions incompatibles
pour la multiplication"
22        resultat = Matrice.nulle(self.m, other.n)
23        for i in range(self.m):
24            for j in range(other.n):
25                resultat.data[i][j] = sum(
26                    self.data[i][k] * other.data[k][j]
27                    for k in range(self.n)
28                )
29        return resultat
30
31    def transposee(self):
32        """Transposee. Complexite : O(mn)."""
33        return Matrice([[self.data[i][j]
34                          for i in range(self.m)]
35                          for j in range(self.n)])

```

```

36
37     def trace(self):
38         """Trace (matrices carrees). Complexite : O(n)."""
39         assert self.m == self.n, "La trace est definie pour les
matrices carrees"
40         return sum(self.data[i][i] for i in range(self.n))

```

Listing 9: Opérations arithmétiques sur les matrices

5.3 Élimination de Gauss–Jordan

Théorème 5.1 (Élimination de Gauss, cf. Strang [?]). *Toute matrice $A \in M_n(\mathbb{R})$ inversible se décompose en $A = LU$ où L est triangulaire inférieure à diagonale unitaire et U triangulaire supérieure. L'élimination de Gauss calcule cette décomposition en $\mathcal{O}(n^3)$ opérations.*

```

1     def gauss_jordan(self, b=None):
2         """
3         Elimination de Gauss-Jordan avec pivot partiel.
4
5         Si b est fourni, resout Ax = b et retourne x.
6         Sinon, retourne la forme echelonnee reduite (RREF).
7
8         Complexite : O(n^3).
9
10        Reference : Trefethen & Bau, Numerical Linear Algebra,
11        Lecture 20 (stabilite de la factorisation
LU).
12        """
13        import copy
14        n = self.m
15        # Matrice augmentee [A | b] ou juste A si b est None
16        if b is not None:
17            M = [row[:] + [b[i]] for i, row in enumerate(self.
data)]
18            ncols = n + 1
19        else:
20            M = [row[:] for row in self.data]
21            ncols = self.n
22
23        # Phase de descente (elimination)
24        pivot_col = 0
25        for pivot_row in range(n):
26            # Pivot partiel : cherche le plus grand element en
valeur absolue
27            max_val = abs(M[pivot_row][pivot_col])
28            max_row = pivot_row
29            for k in range(pivot_row + 1, n):
30                if abs(M[k][pivot_col]) > max_val:
31                    max_val, max_row = abs(M[k][pivot_col]), k

```

```

32         if max_val < 1e-12: # colonne nulle : passe au
suivant
33             pivot_col += 1
34             if pivot_col >= ncols:
35                 break
36             continue
37         M[pivot_row], M[max_row] = M[max_row], M[pivot_row]
38         # Normalisation du pivot
39         pivot = M[pivot_row][pivot_col]
40         M[pivot_row] = [x / pivot for x in M[pivot_row]]
41         # Elimination dans toutes les autres lignes
42         for i in range(n):
43             if i != pivot_row and abs(M[i][pivot_col]) > 1e
-12:
44                 facteur = M[i][pivot_col]
45                 M[i] = [M[i][j] - facteur * M[pivot_row][j]
46                     for j in range(ncols)]
47             pivot_col += 1
48
49         if b is not None:
50             return [M[i][-1] for i in range(n)] # solution x
51         return Matrice([row[:self.n] for row in M]) # RREF
52
53     def determinant(self):
54         """
55         Determinant par elimination de Gauss avec pivot partiel
56         .
57         Complexite : O(n^3).
58         """
59         assert self.m == self.n, "Le determinant est defini
pour les matrices carrees"
60         n = self.m
61         M = [row[:] for row in self.data]
62         det = 1.0
63         for col in range(n):
64             # Pivot partiel
65             max_row = max(range(col, n), key=lambda r: abs(M[r
][col]))
66             if abs(M[max_row][col]) < 1e-12:
67                 return 0.0
68             if max_row != col:
69                 M[col], M[max_row] = M[max_row], M[col]
70                 det *= -1
71             det *= M[col][col]
72             for i in range(col + 1, n):
73                 facteur = M[i][col] / M[col][col]
74                 for j in range(col, n):
75                     M[i][j] -= facteur * M[col][j]
76         return det

```

Listing 10: Élimination de Gauss avec pivot partiel

Exemple 5.1 (Résolution d'un système linéaire). Résoudre $Ax = b$ avec

$$A = \begin{pmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{pmatrix}, \quad b = \begin{pmatrix} 8 \\ -11 \\ -3 \end{pmatrix}.$$

La solution exacte est $x = (2, 3, -1)$.

```
1 A = Matrice([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]])
2 b = [8, -11, -3]
3 x = A.gauss_jordan(b)
4 print(x)      # [2.0, 3.0, -1.0]
5 print(A.determinant()) # -1.0
```

6 Introduction à Lean 4 : Formalisation des Polynômes

6.1 Introduction

Lean 4 est un assistant de preuve qui permet de *certifier* les propriétés mathématiques que Python ne peut que *tester*. La bibliothèque `Mathlib` fournit déjà une formalisation complète de $\mathbb{K}[X]$ via `Polynomial` [?].

Nous allons écrire quelques lemmes élémentaires pour prendre en main la syntaxe.

6.2 Structure de base en Lean 4

```
1 import Mathlib.RingTheory.Polynomial.Basic
2 import Mathlib.Data.Polynomial.Degree.Definitions
3
4 open Polynomial
5
6 -- Type : polynomes a coefficients reels
7 -- R[X] est note Polynomial R dans Mathlib
8
9 -- Lemme 1 : le degre de la somme est borne
10 -- deg(P + Q) <= max(deg P, deg Q)
11 -- Ce resultat est deja dans Mathlib :
12 #check Polynomial.natDegree_add_le
13
14 -- Lemme 2 : commutativite de l'addition
15 -- Ce resultat decoule du fait que R[X] est un anneau
   commutatif
16 example (P Q : Polynomial Real) : P + Q = Q + P := by
17   ring
18
19 -- Lemme 3 : evaluation de la derivee en un point
20 -- (P')(x) = derivee formelle evaluee en x
21 -- La derivee formelle est Polynomial.derivative dans Mathlib
22 #check Polynomial.derivative
23
24 -- Lemme 4 : Horner (version symbolique)
```

```

25 -- On peut exprimer que l'évaluation est un homomorphisme d'
    anneaux
26 -- Polynomial.eval est l'évaluation : Polynomial R -> R -> R
27 example (P Q : Polynomial Real) (x : Real) :
28   (P + Q).eval x = P.eval x + Q.eval x := by
29   simp [Polynomial.eval_add]

```

Listing 11: Premiers lemmes sur les polynômes en Lean 4

```

1 -- Demonstration guidée : deg(P + Q) <= max(deg P, deg Q)
2 -- Ceci est Polynomial.natDegree_add_le dans Mathlib.
3 -- Voici comment le dériver manuellement pour des coefficients
    dans ZZ.
4
5 example (P Q : Polynomial Int) :
6   (P + Q).natDegree <= max P.natDegree Q.natDegree := by
7   exact Polynomial.natDegree_add_le P Q
8
9 -- Exercice Lean : montrer que (X + 1)^2 = X^2 + 2*X + 1
10 example : (Polynomial.X + 1 : Polynomial Int) ^ 2 =
11   Polynomial.X ^ 2 + 2 * Polynomial.X + 1 := by
12   ring

```

Listing 12: Preuve de la borne sur le degré de la somme

Remarque 6.1. Lean 4 avec Mathlib couvre l'intégralité de l'algèbre des polynômes (anneaux de polynômes, factorisation, corps de fractions...). La tâche du mathématicien est de connaître la bibliothèque pour s'y retrouver, et de savoir formaliser les preuves nouvelles. Une bonne entrée en matière est le tutoriel officiel *Mathematics in Lean* [?], section 8.

7 Analyse de Complexité

7.1 Tableau récapitulatif

Algorithme	Complexité	Référence
Addition $P + Q$	$\mathcal{O}(\max(m, n))$	trivial
Évaluation naïve $P(x)$	$\mathcal{O}(n^2)$	n termes, chacun en $\mathcal{O}(n)$
Évaluation de Horner $P(x)$	$\mathcal{O}(n)$	CLRS ch. 2, exercice 2-3
Multiplication $P \cdot Q$	$\mathcal{O}(m \cdot n)$	algorithme naïf
PGCD(P, Q) (Euclide)	$\mathcal{O}(n^2)$	n étapes en $\mathcal{O}(n)$
Division euclidienne	$\mathcal{O}(m \cdot n)$	division longue
Addition matricielle $A + B$	$\mathcal{O}(mn)$	trivial
Multiplication $A \cdot B$	$\mathcal{O}(mnp)$	naïf ; $\mathcal{O}(n^{2.37})$ Coppersmith–Winograd
Élimination de Gauss	$\mathcal{O}(n^3)$	Trefethen–Bau, Lecture 20
Déterminant (Gauss)	$\mathcal{O}(n^3)$	idem

7.2 Master Theorem et récurrences

Pour l'exponentiation rapide de polynômes (P^n) :

$$T(n) = T(n/2) + \mathcal{O}(d^2) \implies T(n) = \mathcal{O}(d^2 \log n),$$

où $d = \deg(P)$ et $\mathcal{O}(d^2)$ est le coût d'une multiplication.

8 Exercices

8.1 Exercices sur la POO

Exercice 8.1 (Classe `Complexe`). Implémentez une classe `Complexe` représentant un nombre $z = a + ib$ avec : les méthodes `__add__`, `__mul__`, `conjugue()`, `module()` et `__str__`. Vérifiez que $|z \cdot w| = |z| \cdot |w|$.

Exercice 8.2 (Extension de `Polynome`). Ajoutez à la classe `Polynome` :

- (a) Une méthode `integrer(c)` qui calcule la primitive $\int P$ de constante c .
- (b) La surcharge de `__call__` afin de pouvoir écrire `P(x)` directement.
- (c) La méthode `__iter__` pour itérer sur les coefficients.

Exercice 8.3 (Interpolation de Lagrange). Étant donné $n+1$ points $(x_0, y_0), \dots, (x_n, y_n)$, le polynôme d'interpolation de Lagrange est

$$L(X) = \sum_{i=0}^n y_i \prod_{j \neq i} \frac{X - x_j}{x_i - x_j}.$$

Implémentez une fonction `lagrange(xs, ys)` qui retourne un objet `Polynome`. Testez avec les points $(0, 0), (1, 1), (2, 4)$ (résultat attendu : X^2).

Exercice 8.4 (Vérification des axiomes d'anneau). Écrivez une fonction `verifier_anneau(P, Q, R)` qui vérifie pour des objets `Polynome` donnés les axiomes suivants :

- Commutativité de $+$: $P + Q = Q + P$.
- Associativité de $\times$: $(P \cdot Q) \cdot R = P \cdot (Q \cdot R)$.
- Distributivité : $P \cdot (Q + R) = P \cdot Q + P \cdot R$.

8.2 Exercices sur les matrices

Exercice 8.5 (Méthode de Jacobi). L'itération de Jacobi pour résoudre $Ax = b$ met à jour

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right).$$

Ajoutez une méthode `jacobi(b, tol=1e-8, max_iter=1000)` à la classe `Matrice`. Comparez sa vitesse avec Gauss pour des matrices aléatoires 100×100 .

Exercice 8.6 (Décomposition LU). Implémentez la décomposition $A = LU$ (sans pivot) comme méthode de la classe `Matrice`. Exprimez ensuite la résolution de $Ax = b$ en deux étapes : $Ly = b$ puis $Ux = y$ (substitutions avant/arrière).

8.3 Exercice Lean 4

Exercice 8.7 (Lean 4 – Polynôme constant). Vérifiez en Lean 4 que le polynôme $X^2 - 1$ se factorise en $(X - 1)(X + 1)$.

```
1 example : (Polynomial.X ^ 2 - 1 : Polynomial Int) =  
2           (Polynomial.X - 1) * (Polynomial.X + 1) := by  
3   ring
```

Modifiez l'exemple pour traiter $X^3 - 1$.

9 Problèmes

Problème 9.1 (Multiplication rapide de polynômes via FFT). La multiplication naïve de deux polynômes de degré n coûte $\mathcal{O}(n^2)$. L'algorithme basé sur la Transformée de Fourier Rapide (FFT) la ramène à $\mathcal{O}(n \log n)$, en exploitant le fait que l'évaluation en $2n$ racines de l'unité peut se faire en $\mathcal{O}(n \log n)$.

- (a) Expliquez le principe (évaluation, multiplication point par point, interpolation).
- (b) Implémentez une version utilisant `numpy.fft.fft` et `ifft`.
- (c) Comparez empiriquement avec la multiplication naïve pour $n = 10, 100, 1000$.

Référence : CLRS, chapitre 30.

Problème 9.2 (Classe `MatriceCreuse`). Une matrice *creuse* (sparse) a la plupart de ses entrées nulles. Le stockage naïf en $\mathcal{O}(n^2)$ est inefficace pour de grandes matrices.

- (a) Implémentez une classe `MatriceCreuse` héritant de `StructureAlgebrique`, en stockant uniquement les entrées non nulles dans un dictionnaire `{(i,j): valeur}`.
- (b) Implémentez l'addition et la multiplication matrice-vecteur.
- (c) Comparez l'utilisation mémoire avec la classe `Matrice` dense.

Référence : Strang, *CSE*, chapter 7.

Problème 9.3 (Corps de polynômes : $\mathbb{F}_p[X] / (P(X))$). Soit p premier et $P \in \mathbb{F}_p[X]$ irréductible de degré n . L'anneau quotient $\mathbb{F}_p[X] / (P(X))$ est un corps à p^n éléments (corps de Galois $\text{GF}(p^n)$).

- (a) Implémentez une classe `PolyMod` représentant un élément de $\mathbb{F}_2[X] / (X^8 + X^4 + X^3 + X + 1)$ (AES utilise ce corps).
- (b) Implémentez l'addition et la multiplication dans ce corps.
- (c) Calculez l'inverse d'un élément non nul par l'algorithme d'Euclide étendu.

Référence : CLRS, section 31.7 ; FIPS 197 (standard AES).

10 Livrable

Livrable étudiant

Enrichissez votre dépôt GitHub M1-Informatique avec :

- Un notebook `seance3.ipynb` contenant :
 - La classe `Polynome` complète avec tests.
 - La classe `Matrice` avec `gauss_jordan` et `determinant`.
 - Les mesures empiriques de complexité (Horner vs naïf, Gauss).
 - Au moins 3 exercices résolus parmi ceux proposés.
 - Le fichier `seance3.lean` avec les exercices Lean.
- Un commit : "Séance 3 : P00 avancée, Polynômes et Matrices".

Critères d'évaluation (10 points) :

- Correction et complétude des classes `Polynome` et `Matrice` (4 pts)
- Analyse de complexité et mesures empiriques (2 pts)
- Exercices Lean 4 (2 pts)
- Documentation, tests, utilisation de Git (2 pts)

References

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms*, 4e éd. MIT Press, 2022. Chapitres 2 (Horner), 30 (FFT), 31 (arithmétique modulaire).
- [2] Chambert-Loir, A., et al. *Exo7 – Algorithmes et mathématiques*. <http://exo7.emath.fr>, 2020.
- [3] Strang, G. *Computational Science and Engineering*. Wellesley–Cambridge Press, 2007. Chapters 1–3 (élimination de Gauss, décomposition LU).
- [4] Trefethen, L. N. & Bau, D. *Numerical Linear Algebra*. SIAM, 1997. Lectures 20–23 (factorisation LU, stabilité).
- [5] Avigad, J., et al. *Mathematics in Lean*. https://leanprover-community.github.io/mathematics_in_lean/, 2024. Section 8 : polynômes et corps en Lean 4.
- [6] Avigad, J., de Moura, L., Kong, S., & Ullrich, S. *Theorem Proving in Lean 4*. https://leanprover.github.io/theorem_proving_in_lean4/, 2024.
- [7] The Mathlib Community. *Mathlib4 – Mathematics Library for Lean 4*. https://leanprover-community.github.io/mathlib4_docs/Mathlib/RingTheory/Polynomial/Basic.html, 2024.
- [8] Sedgewick, R. & Wayne, K. *Algorithms*, 4e éd. Addison-Wesley, 2011. Chapter 2.2 (tri fusion), Chapter 9 (tables de hachage).